

BÜYÜK DİL MODELLERİ (LLM) ve TOKEN KAVRAMI

Temel Kavramlar, Çalışma Mantığı ve Uygulama Örnekleri

Hazırlayan: Yasemin KUŞ

Staj Sunumu

Temmuz 2026

1. Giriş: LLM Nedir?

Büyük Dil Modeli (Large Language Model - LLM), devasa miktarda metin verisiyle eğitilmiş, insan dilini anlama ve üretme yeteneğine sahip yapay zeka modelleridir. GPT, Claude, Gemini ve LLaMA gibi modeller bu kategoriye girer.

Bu modeller, kendisine verilen bir metni (prompt) analiz ederek bir sonraki en olası kelimeyi/parçayı tahmin etme prensibiyle çalışır. Ancak modeller kelimelerle değil, 'token' adı verilen daha küçük birimlerle işlem yapar.

LLM'ler için Tokenizasyon Neden Önemlidir?

Doğal insan dili karmaşıktır ve birçok farklı değişkenle doludur ve LLM'ler büyük veri işleme modelleridir: Bizim gibi düşünmez, hatırlamaz veya bilgiyi işlemezler. Bunun yerine, veri kalıplarını inceler ve büyük miktarda veriye dayanarak en mantıklı çıktıyı oluştururlar. Bunu yapmak için, LLM milyarlarca kelime öbeğini işlemelidir. Metni daha küçük ve daha tahmin edilebilir birimlere ayırarak, LLM verilerdeki kalıpları çok daha verimli bir şekilde analiz edebilir, işleyebilir ve tespit edebilir.

2. Token Nedir?

Token, bir metnin model tarafından işlenebilecek en küçük anlamlı parçasıdır. Bir token; bir kelime, bir kelimenin parçası (alt-kelime), tek bir karakter veya bir noktalama işareti olabilir.

LLM'ler metni doğrudan harf veya kelime olarak değil, önceden belirlenmiş bir 'kelime dağarcığı' (vocabulary) içindeki token ID'leri olarak işler.

2.1. Neden Kelime Değil de Token?

- Kelime bazlı yaklaşım, dildeki sonsuz sayıda kelime, çekim eki ve yeni terim (örn. marka isimleri) nedeniyle çok büyük bir sözlük gerektirir.
- Karakter bazlı yaklaşım ise diziyi çok uzattığı için işlem maliyetini artırır ve anlamsal bağlamı zayıflatır.
- Token bazlı yaklaşım (alt-kelime tokenizasyonu), bu ikisi arasında bir denge kurar: hem sözlük boyutunu makul tutar hem de bilinmeyen kelimeleri parçalara ayırarak işleyebilir.

3. Tokenization (Tokenleştirme) Süreci

Tokenization, ham metnin sayısal token ID dizisine dönüştürülmesi işlemidir. Bu süreç modelin eğitim aşamasında belirlenen bir 'tokenizer' (tokenleştirici) tarafından gerçekleştirilir.

3.1. İşlem Adımları

- Metin normalize edilir (küçük/büyük harf, boşluk düzenlemeleri vb.).
- Metin, tokenizer'ın öğrendiği alt-kelime birimlerine göre parçalanır.
- Her token, sözlükteki karşılığı olan benzersiz bir sayısal ID'ye (Token ID) çevrilir.
- Model bu ID dizisini girdi (input) olarak alır ve çıktı olarak yine token ID dizisi üretir.
- Üretilen ID dizisi, tokenizer tarafından tekrar okunabilir metne çevrilir (decode).

3.2. Örnek Tokenizasyon

Hello, welcome to your internship. [13225, 11, 12591, 316, 634, 53792, 13]

ŞEKİL.1 TOKENİZASYON ÖRNEĞİ

My internship started today. [5444, 53792, 5424, 4044, 13]

ŞEKİL.2 TOKENİZASYON ÖRNEĞİ

I was born in July. [40, 673, 12275, 306, 8170, 13]

ŞEKİL.3 TOKENİZASYON ÖRNEĞİ

July is much hotter than June. [30552, 382, 2009, 140032, 1572, 7843, 13]

ŞEKİL.4 TOKENİZASYON ÖRNEĞİ

4. Yaygın Tokenizer Algoritmaları

Farklı LLM aileleri, farklı tokenizasyon algoritmaları kullanır. En yaygın üç yöntem:

Yöntem	Kısa Açıklama	Kullanan Modeller (Örnek)
BPE (Byte Pair Encoding)	En sık birlikte görülen karakter/karakter çiftlerini birleştirerek sözlük oluşturur.	GPT ailesi
WordPiece	Kelimelerin sadece yan yana gelme olasılığına bakmaz, parçalar birleştiğinde dil modelinin genel olasılığına da bakar.	BERT
SentencePiece / Unigram	Ham metni doğrudan işler, dil bağımsız çalışabilir. Kayıpsız çalışır.	T5, LLaMA, Gemini

5. Token Sayısı Neden Önemlidir?

5.1. Context Window (Bağlam Penceresi)

Her LLM'in işleyebileceği maksimum token sayısı sınırlıdır. Buna 'context window' denir. Bu sınır; hem kullanıcının girdisini (prompt) hem de modelin ürettiği çıktıyı (completion) kapsar.

- Örneğin bir modelin context window'u 128.000 token ise; girdi + çıktı toplamı bu sınırı aşamaz.
- Uzun belgeler, sohbet geçmişi veya kod dosyaları context window'u hızla doldurabilir.
- Sistem verimliliği için prompt'ları mümkün mertebe optimize etmek ve gereksiz token harcamamak gerekir.

5.2. Performans ve Hız

Daha fazla token, modelin daha fazla işlem yapması anlamına gelir; bu da yanıt süresini uzatabilir. Verimli prompt yazımı (gereksiz tekrarlardan kaçınma) hem maliyeti hem de gecikmeyi azaltır.

6. LLM Mimarisine Kısa Bakış: Transformer

Günümüz LLM'lerinin büyük çoğunluğu 'Transformer' mimarisine dayanır. Bu mimari, 2017 yılında yayımlanan "Attention Is All You Need" makalesiyle tanıtılmıştır.

- Girdi: Token ID dizisi, önce 'embedding' katmanında sayısal vektörlere dönüştürülür.
- Positional Encoding: Tokenların cümle içindeki sırası modele ayrıca bildirilir.
- Self-Attention: Model, her tokenın diğer tokenlarla olan ilişkisini (bağlamsal önemini) hesaplar.
- Çıktı: Model, bir sonraki en olası tokenın olasılık dağılımını üretir ve bu süreç token token tekrarlanarak metin üretilir.

7. Basit Bir Uygulama Örneği (Python)

Aşağıda, OpenAI uyumlu bir tokenizer kütüphanesi (tiktoken) ile bir metnin kaç token olduğunu hesaplayan basit bir örnek verilmiştir:

```
import tiktoken

encoder = tiktoken.encoding_for_model("gpt-4")
tokens = encoder.encode("Merhaba, staj sürecine hoş geldin!")
print(len(tokens)) # örn. çıktı: 11

print(encoder.decode(tokens)) # metni geri döndürür
```

Bu tarz bir kontrol, API'ye gönderilecek isteğin token limitini aşp aşmadığını göndermeden önce test etmeyi sağlar.

8. Özet

- LLM, metni token adı verilen küçük birimler halinde işleyen bir yapay zeka modelidir.
- Tokenizasyon, ham metni sayısal token ID'lerine çeviren ön işlem adıdır.
- BPE, WordPiece ve SentencePiece en yaygın tokenizasyon yöntemleridir.
- Token sayısı; context window sınırını, API maliyetini ve yanıt hızını doğrudan etkiler.
- Transformer mimarisi, tokenlar arasındaki bağlamsal ilişkiyi 'attention' mekanizmasıyla hesaplar.